MikroTik Configuration Guide

Provision and connect MikroTik RouterOS devices over WireGuard

Blackie Networks

WireGuard VPN Management Platform · docs.blackie-networks.com
Generated 2026-08-18

MikroTik Integration Overview

Blackie Networks can auto-provision a MikroTik RouterOS device as a WireGuard peer and generate a ready-to-run RouterOS script that configures the router end-to-end: interface, keys, address, peer, route, and a connectivity self-test.

> **NOTE**
> Requires RouterOS v7.0 or later — native WireGuard support was introduced in v7.

Two ways to provision a router

- Dashboard-managed router: POST /api/routers (authenticated) creates a WireGuard peer, allocates three public TCP proxy ports (Winbox, SSH, API), starts the TCP proxy, and tracks the router for billing. This is the path used by the user-facing app.
- Quick script generation: the short URL GET /mt/:name generates (or reuses) a client and returns a RouterOS .rsc script directly — useful for scripting or one-off routers outside the billed router flow.

Method 1 — Quick script via curl

```
# Replace YOUR_SERVER with your API host, and "office-router" with a name
curl -sS "http://YOUR_SERVER:5000/mt/office-router" -o office-router.rsc

# Optional query parameters
curl -sS "http://YOUR_SERVER:5000/mt/office-router?iface=wg-office" -o office-router.rsc
curl -sS "http://YOUR_SERVER:5000/mt/office-router?notes=Nairobi%20Branch" -o office-router.rsc
```

Method 2 — Full options via POST

```
curl -X POST http://YOUR_SERVER:5000/generate-mikrotik \
  -H "Content-Type: application/json" \
  -d '{
    "name": "office-router",
    "notes": "Nairobi Branch",
    "interfaceName": "wireguard-office",
    "allowedSubnet": "10.0.0.0/24"
  }' \
  -o office-router.rsc
```

Method 3 — Fetch and run directly from RouterOS

RouterOS can pull and execute the script itself, with no file leaving the router:

```
/tool fetch url="http://YOUR_SERVER:5000/mt/office-router" dst-path=wg-config.rsc
/system script add name=wg-setup source=[/file get wg-config.rsc contents]
/system script run wg-setup
```

The generated script will:

- Create a /interface wireguard interface and set its private key and listen port.
- Assign the allocated VPN address (e.g. 10.0.0.7/32).
- Add the server as a peer, with its public key, endpoint, allowed-address and a keepalive.
- Add a route into the VPN subnet.
- Ping the server (10.0.0.1) to confirm the tunnel came up.

Verifying the connection

```
/interface wireguard print
/ip address print where interface~"wireguard"
/ping 10.0.0.1 count=5
```

A working setup shows the WireGuard interface as running, a 10.0.0.x/32 address assigned, and successful replies from the ping to 10.0.0.1.

Reaching Winbox, SSH and the RouterOS API remotely

Routers provisioned through the dashboard (POST /api/routers) get three public TCP ports forwarded through to the router's VPN address, so you can reach it without being on the VPN yourself:

Service	Router-side port
`Winbox`	8291
`SSH`	22
`RouterOS API`	8728

Public-facing ports are allocated from fixed ranges per service — Winbox from 1000–33
6666, and the API from 6667–9999 — and are shown for each router in the dashboard'

Health monitoring

The server polls each provisioned router every few minutes over SSH, using a dedicate
RouterOS user (credentials set via MIKROTIK_SYSTEM_USERNAME /
MIKROTIK_SYSTEM_PASSWORD) restricted to the 10.0.0.0/24 VPN subnet, to record
memory for the dashboard.

Managing existing clients

```
# List all clients
curl http://YOUR_SERVER:5000/clients

# Re-download an existing client's RouterOS script
curl -sS "http://YOUR_SERVER:5000/mt/office-router" -o office-router.rsc

# Re-download the same client as a standard WireGuard .conf
curl -sS "http://YOUR_SERVER:5000/clients/office-router" -o office-router.conf

# Remove a client
curl -X DELETE http://YOUR_SERVER:5000/clients/office-router
```

Troubleshooting

- "Client with this name already exists" — pick a different name, or delete the existing client first.
- Script does nothing in RouterOS — confirm RouterOS v7.0+, and that the .rsc file downloaded in full (not truncated).
- Router can't reach the server — verify SERVER_ENDPOINT is correct, that UDP 51820 is open on both ends, and check firewall rules on the router.